

Solución - Trabajo Práctico N°3

Clean Code, Refactoring y Code Review

Problemas Detectados

Problema	Descripción
Nombres poco claros	Variables y métodos con nombres ambiguos.
Muchas responsabilidades	Toda la lógica está concentrada en un único método.
Baja legibilidad	El código es difícil de entender.
Primitive Obsession	Uso excesivo de boolean y tipos primitivos.
Falta de cohesión	Se mezclan múltiples reglas de negocio.

Introduce Parameter Object

```
public class DatosFacturacion {  
  
    private final boolean pacientePremium;  
    private final boolean tieneObraSocial;  
    private final int cantidadEstudios;  
  
    public DatosFacturacion(  
        boolean pacientePremium,  
        boolean tieneObraSocial,  
        int cantidadEstudios  
    ) {  
        this.pacientePremium = pacientePremium;  
        this.tieneObraSocial = tieneObraSocial;  
        this.cantidadEstudios = cantidadEstudios;  
    }  
  
    public boolean isPacientePremium() {  
        return pacientePremium;  
    }  
  
    public boolean isTieneObraSocial() {  
        return tieneObraSocial;  
    }  
  
    public int getCantidadEstudios() {  
        return cantidadEstudios;  
    }  
}
```

Clase Refactorizada

```
import java.util.List;  
  
public class FacturacionService {  
  
    public double calcularTotal(  
        List<Double> estudios,  
        DatosFacturacion datos  
    ) {  
  
        validarEstudios(estudios);  
        double subtotal =
```

```

        calcularSubtotal(estudios, datos);

    subtotal =
        aplicarDescuentoObraSocial(
            subtotal,
            datos
        );

    subtotal =
        aplicarBonificacionCantidad(
            subtotal,
            datos
        );

    return subtotal;
}

private void validarEstudios(List<Double> estudios) {

    if (estudios == null || estudios.isEmpty()) {
        throw new IllegalArgumentException(
            "La lista de estudios no puede estar vacia"
        );
    }

}

}

```

Refactorings Aplicados

Técnica	Aplicación
Extract Method	Separación de cálculos complejos.
Introduce Parameter Object	Creación de la clase DatosFacturacion.
Move Method	Separación de responsabilidades entre métodos.

Mejoras Obtenidas

- Mayor cohesión.
- Menor acoplamiento.
- Código más legible.
- Mayor facilidad de testing.
- Mejor mantenibilidad y extensibilidad.

Checklist de Code Review

Ítem	Resultado
Nombres descriptivos	Correcto
Funciones pequeñas	Correcto
Duplicación	No detectada
Manejo de errores	Adecuado
Cohesión	Alta
Acoplamiento	Bajo
Legibilidad	Correcta

Feedback Profesional

La versión original presentaba múltiples responsabilidades dentro de un único método, dificultando su comprensión y mantenimiento. La refactorización permitió separar claramente cada regla de negocio, mejorando la cohesión y reduciendo el acoplamiento. Además, la introducción de un Parameter Object incrementó significativamente la claridad semántica del código.